

CAPSTONE + ROADMAP · DAY 3 下午 · 150 MIN

三天前你有一个空目录——现在你有什么？

DAY 1

AgentOS 全景 + Knowledge 模块

DDD 4 文档 · 三层治理 · Capture+Retrieve hooks · Health check

Agent 有了「记忆」

DAY 2

Delivery Engine + SDLC 实弹

Stages · Gates · Profiles · State + 跑完一个 feature 全流程

Agent 有了「纪律」

DAY 3

验证 + Loop Engineering

Behavioral Contract · Eval script · Loop config

Agent 有了「自我检测」与「自动驾驶」

三件事加起来 = 一个可运转的 AgentOS —— 你的判断被编译成了系统。

你的 my-agentos/ 应该长这样

```
my-agentos/
├── README.md                ← 使用说明 (Capstone 产出)
├── knowledge/
│   ├── PRODUCT.md / TECH.md / IMPROVEMENT.md / PROJECT.md ← DDD
│   └── health.sh           ← 健康检查
├── governance/
│   ├── principles.md       ← 3-5 条原则 (有优先级)
│   ├── rules/ R001-*.md _retired/
│   └── gates/ check-*.sh _graduated/
├── engine/
│   ├── SKILL.md stages.md gates.md profiles.md
│   └── STATE.md loop-config.md
├── eval/ golden-set.md run-eval.sh
├── hooks/ on-session-start.sh on-session-end.sh
├── skills/ distill/ gate-review/ (可选)
├── spec/ recognize/        ← Engine 产出 artifacts
├── corrections.log         ← 蒸馏原料
├── ci/ verify.sh           ← 业务验证
└── docs/ 30-60-90-action-plan.md ← Roadmap (Capstone 产出)
```

Capstone 任务

1

补缺失部分

对照结构补全

2

写 README.md

使用说明

3

跑一次 eval

记录当前得分

4

写 action plan

30/60/90

45 分钟整合——补缺 + README + Eval

1

结构补全 15 min

对照结构检查缺失文件 · hooks 注册到 settings.json · eval/run-eval.sh 可执行

2

写 README.md 15 min

是什么 · 核心 Principles · 使用方式 · 健康检查 · 行为验证

3

跑 Eval 10 min

执行 run-eval.sh · 记录得分 · 识别最低分维度

4

Git commit 5 min

git init + 首次 commit = 你的 AgentOS v1.0

My AgentOS

是什么
[一句话: AgentOS 核心理念]

核心 Principles
[列出你的 3-5 条原则]

使用方式

1. 克隆 repo 到项目根目录
2. 配置 hooks (settings.json)
3. 启动 session – Knowledge 注入
4. 下发任务 – Engine 驱动

健康检查 / 行为验证

```
bash knowledge/health.sh
bash eval/run-eval.sh
```

看看邻座的 AgentOS——和你有什么不同

方式：2-3 人一组，交换 repo（或屏幕分享）· 15 分钟审查 + 5 分钟反馈 · 不是评对错，是讨论「为什么不同」

维度	怎么看	信号
Principles 可判定性	对每条做「反例测试」——能否想到一个场景它无法明确告诉你做到没？	能想到 = 不够可判定
Gates 合理性	gate 级别选得对吗？太重 / 太轻？	全 L3=没收益；全 L1=形同虚设
覆盖度	随机抽 3 条 correction——能追溯到哪条 principle/rule？	追溯不到 = 覆盖缺口
蒸馏度	rules 数量合理吗？有退休的吗？governance 行数 / corrections 比例？	只增不减 = 没蒸馏
差异点	最大的差异是什么？为什么不同？有想借鉴的吗？	差异 = 品味的多样性

同一个错题本，N 种 AgentOS——你看到了什么

收集互评发现（全班讨论）

- 最让你惊讶的差异是什么？
- 你发现了自己的什么盲点？
- 有没有对方的某个设计你想借鉴的？

预期讨论点

- Principles 优先级排序差异大
- Gates 级别差异大（全 L1 激进 vs 全 L3 保守）
- corrections 数量 / 质量差异（20+ vs 5-6）
- 蒸馏程度差异（有人做了、有人没做完）

教学总结

差异是正常的

不同人面对同一系统做出不同判断

每个判断都应有理由

说不出为什么选 = 「没选」，不是「选了」

互评最大价值

拓展你的判断维度 —— 有理由的差异是品味，没理由的是随意

三天是开始——接下来怎么走



动手：30/60/90 行动计划

Lab 任务 (15 min)

- 创建 docs/30-60-90-action-plan.md
- 30 天：3-5 个具体行动 + 成功指标
- 60 天：3-5 个具体行动 + 成功指标
- 90 天：3-5 个具体行动 + 成功指标
- 识别第一个「回去就做」的事情

最重要的一个问题

明天回到办公室，你第一件做的事情是什么？

「配 hooks」= 具体，好；「学更多」= 不够，你已经够了，要的是做。

30/60/90 Action Plan

30 Days: 运行 + 积累

- [] 将 AgentOS 接入项目 [名]
 - [] 跑 3 个 feature 通过 Engine
 - [] 积累 50+ corrections
 - [] 做 2 次蒸馏 (第15、30天)
- 成功指标: principles $\leq$ 5, rules 有退休

60 Days: 渐进 + Loop

- [] G1/G3 降为 L1
 - [] 第一次 bugfix loop 实验
 - [] PROJECT.md 自动更新
- 成功指标: 至少 1 类任务可 loop

90 Days: 进化 + 迁移

- [] 接入第二个项目
 - [] Principles 跨项目复用
 - [] Governance 总量 < Day30 的 70%
- 成功指标: 新项目接入 < 1 天

课程终章 · GOOD LUCK. BUILD ON.

你带走的不是错题本——是肌肉

错题本 = 杠铃（工具） **AgentOS = 肌肉（带走的）** 课程 = 健身房（练习场）



判断编译

把品味变成 agent 跑不掉的约束



蒸馏思维

做减法 > 做加法



三件套模子

人站哪 → 品味是什么 → 编译成什么



系统思维

不是配 rules，是建会进化的系统

工具会变，模型会换。你的判断——只会越来越锋利。

祝你们的 AgentOS 越用越短。 Good luck. Build on.